

Real-Time Bangladeshi Taka Banknote Detection

A YOLOv8 Object-Detection System Trained on Synthetically Composited Data with Real-World Backgrounds

Detailed Technical Project Report

Deep Learning · Computer Vision · Object Detection

Figure 2 — YOLOv8s Detection Architecture



11.1M parameters · 28.5 GFLOPs · 7.3 ms/image inference (RTX 4060)

Prepared: July 2026

Abstract

This project develops a real-time detection system that localises and classifies the nine denominations of Bangladeshi Taka (BDT) paper currency — 2, 5, 10, 20, 50, 100, 200, 500 and 1000 Taka — from a live camera feed. A recurring problem with publicly available BDT datasets is that they consist exclusively of clean, close-up, full-frame note scans, which teach a model that “the entire frame is the note” and fail on a webcam, where a note appears at an arbitrary scale, angle and position against cluttered real backgrounds. To bridge this synthetic-to-real domain gap, we designed a copy-paste compositing pipeline that pastes 5,073 background-free note images onto 1,500 real-world photographs from the COCO dataset, applying perspective warp, rotation, scale variation, colour/brightness jitter, motion blur, sensor noise and JPEG re-compression, and automatically generating exact bounding-box labels. The resulting 22,315-image dataset was used to fine-tune a COCO-pretrained YOLOv8s detector for 80 epochs. On a held-out test set of 2,238 images the model achieves mAP@0.5 of 0.995, mAP@0.5:0.95 of 0.847, precision of 0.998 and recall of 0.997, at 7.3 ms per image (≈ 135 FPS) on an NVIDIA RTX 4060 Laptop GPU. The system runs live from a standard webcam and includes automatic low-light brightness correction.

Keywords: Object Detection, YOLOv8, Bangladeshi Currency, Synthetic Data, Domain Randomization, Copy-Paste Augmentation, Transfer Learning, Real-Time Inference.

1. Introduction

Automatic recognition of paper currency has practical value in assistive technology for the visually impaired, automated cash-handling machines, retail self-service and financial automation. For Bangladeshi Taka in particular, an ideal system must not only *classify* which note is present but also *detect* and localise it within a cluttered scene in real time, so that it can operate from a phone or webcam held by a user in an uncontrolled environment.

Most existing BDT recognition work frames the task as *whole-image classification* on clean, centred note images. Such models perform well on their own curated test sets but degrade sharply in the field because real camera frames contain background clutter, perspective distortion, motion blur, variable lighting and notes that occupy only a fraction of the frame. This project deliberately targets that gap by treating the problem as **object detection** and by engineering the training data so that its statistics resemble real webcam imagery rather than studio scans.

Contributions.

- A reproducible synthetic data-generation pipeline that converts a background-free classification dataset into a large, richly augmented detection dataset with automatically generated bounding-box labels.
- A fine-tuned YOLOv8s detector that attains 0.995 mAP@0.5 on a held-out test set while running above 100 FPS on a laptop GPU.
- A deployable real-time webcam application with automatic low-light correction and a TorchScript export for dependency-free inference.

2. Related Work

Early Bangladeshi banknote recognition relied on classical neural networks with hand-engineered features; a neural network using axis-symmetrical masks reported an average accuracy of 98.57% across

eight denominations [4]. More recent approaches use deep convolutional networks and transfer learning: lightweight CNNs and pretrained backbones (MobileNet, NASNetMobile, ResNet152v2, EfficientNet) have been applied to real-time classification for visually impaired users [5,7], and dual-stream MobileNet/EfficientNet architectures have been proposed for robust recognition [7].

For the *detection* formulation, single-stage detectors of the YOLO family are the dominant choice because of their speed. A YOLOv5-based taka detector trained on 3,111 annotated images reported precision and recall of 0.95 and 0.93 [1], and a YOLOv8-nano model with a custom Squeeze-and-Excitation head for multi-currency (USD/EUR/BDT) detection reported mAP@0.5 of 0.972 and mAP@0.5:0.95 of 0.804 at 3.1 ms per image [2]. Our work differs in its explicit focus on closing the synthetic-to-real domain gap through domain-randomised copy-paste compositing, an approach shown to be highly effective for synthetic-to-real object detection [3]. Whereas [1] and [2] annotate real photographs by hand, we generate an order of magnitude more labelled data automatically, with pixel-perfect boxes and full control over the distribution of scales, angles and backgrounds.

3. Dataset and Data Engineering

This section describes the two raw data sources and, in detail, how they were combined into a purpose-built detection dataset. Data engineering is the core contribution of the project, so it is documented step by step.

3.1 Source Currency Dataset (Foreground)

The banknote imagery is drawn from “**A Diverse Image Dataset for Bangladeshi Currency Recognition**” (also published as the *BanglaTaka* dataset), openly available on Mendeley Data [6]. It contains **5,073 background-free images** of BDT paper currency spanning all nine circulating denominations. Images were captured with a variety of mobile-phone cameras and pre-processed with the CamScanner application, giving clean, cropped notes on a plain field. Because each image is already a tight crop of a single note, it is ideal *foreground* material: the whole image can be treated as the object to be pasted.

Denomination	2	5	10	20	50	100	200	500	1000	Total
Source images	445	660	419	480	416	565	423	1243	422	5073

Table 1 — Per-denomination image counts in the source *BanglaTaka* dataset.

3.2 Background Dataset

Real-world backgrounds were sourced from the **COCO val2017** split [8] — a widely used collection of natural photographs containing desks, hands, indoor rooms, streets and outdoor scenes. We downloaded **1,500** images programmatically. Using genuine photographic backgrounds (rather than solid colours or synthetic noise, as in an earlier iteration of this project) is the single most important factor in making the model generalise to real camera frames, because it forces the detector to separate the note from realistic clutter, texture and lighting.

3.3 The Domain-Gap Problem

The reason the two datasets must be combined is a **domain gap**. A model trained only on the background-free *BanglaTaka* images learns that a note fills the entire frame on a plain background. When such a model is shown a webcam frame — where the note is small, rotated, and surrounded by objects — its assumptions break and detection collapses. This was observed directly in an early version of the system, which scored highly on synthetic tests yet failed to detect notes on a live webcam. Compositing the notes onto real backgrounds re-creates deployment conditions at training time and eliminates the gap.

3.4 Copy-Paste Compositing Process

Each source note is pasted onto a randomly chosen, randomly cropped COCO background after a chain of randomised transformations. Crucially, because we control exactly where and at what size the note is placed, the corresponding **bounding box is computed analytically** — no manual annotation is required, and the labels are pixel-perfect. Figure 2 shows the process applied to real assets from our pipeline.

Figure 2 — Copy-Paste Compositing of a Source Note onto a Real COCO Background



Figure 2 — The compositing process on real data: (a) a background-free 100-Taka note and (b) a COCO background are combined; (c) the note is warped, rotated, scaled and colour-jittered; (d) it is pasted at a random location and an exact bounding box (green) with the class label is generated automatically.

Each panel of Figure 2 corresponds to a concrete stage of the algorithm:

- **(a) Source note** — the raw foreground, loaded with an alpha channel so that only the note (not its rectangular border) is pasted.
- **(b) Real background** — a random COCO photo, centre-cropped to a square and resized to 640×640 to form the canvas.
- **(c) Augmented note** — the note after perspective warp, rotation, scaling and photometric jitter, simulating how a real note looks when held at an angle under different lighting.
- **(d) Final composite** — the augmented note pasted at a random position, followed by frame-level effects (blur, noise, JPEG artefacts). The green box is the auto-generated ground-truth label written in YOLO format.

3.5 Bounding-Box Computation

YOLO labels store each box as four normalised numbers — centre-x, centre-y, width and height, each in the range [0,1] relative to the image dimensions. Given a note of pixel size (w, h) pasted at top-left corner (x, y) on a canvas of size S=640, the label is computed directly:

```
x_center = (x + w/2) / S      # normalised horizontal centre
y_center = (y + h/2) / S      # normalised vertical centre
box_w    = w / S              # normalised width
box_h    = h / S              # normalised height

label line -> "<class_id> <x_center> <y_center> <box_w> <box_h>"
```

This closed-form label is the key advantage of synthetic compositing: annotation is free, exact and instantaneous, allowing tens of thousands of labelled examples to be produced automatically.

3.6 Augmentation Operations

The following domain-randomisation operations are applied per composite. Their ranges were chosen to span — and slightly exceed — the variation expected from a hand-held webcam, so that real conditions fall inside the training distribution.

Stage	Operation	Range / Setting	Purpose
Geometry	Perspective warp	70% prob., corner jitter 8%	notes held at an angle
	Rotation	±30°	arbitrary orientation
	Scale (frac. of frame)	0.25 – 0.90	near/far distance
	Random placement	anywhere	off-centre framing
Photometric	HSV jitter	H±18° S×0.5–1.5 V×0.4–1.6	colour/lighting shifts
	Brightness / Contrast	0.4–1.6 / 0.7–1.3	exposure variation
Sensor	Motion blur	20% prob., kernel 3/5/7	hand shake
	Gaussian noise	50% prob., $\sigma \approx 12$	low-light sensor noise
	JPEG re-compression	quality 55–92	webcam compression
Balance	Negative images	10% of output	suppress false positives

Table 2 — Domain-randomisation operations and the real-world variation each targets.

3.7 Final Dataset Composition

Four composites were produced per source note, plus a 10% share of background-only *negative* images (empty labels) that teach the model when *not* to fire. The data was split 80/10/10 **by source note** so that no note appears in more than one split, preventing information leakage between train, validation and test.

Split	Positive Images	Negative Images	Total Images	Note Instances
Train	16,224	1,620	17,844	16,224
Validation	2,016	217	2,233	2,016
Test	2,052	186	2,238	2,052
Total	20,292	2,023	22,315	20,292

Table 3 — Detection dataset composition after synthetic compositing.

4. Methodology and Implementation

The overall workflow is summarised in Figure 3. Three inputs — source notes, real backgrounds and COCO-pretrained YOLOv8s weights — feed the compositing engine, which produces a labelled detection dataset used for training, evaluation and deployment.

Figure 1 — Methodology Pipeline

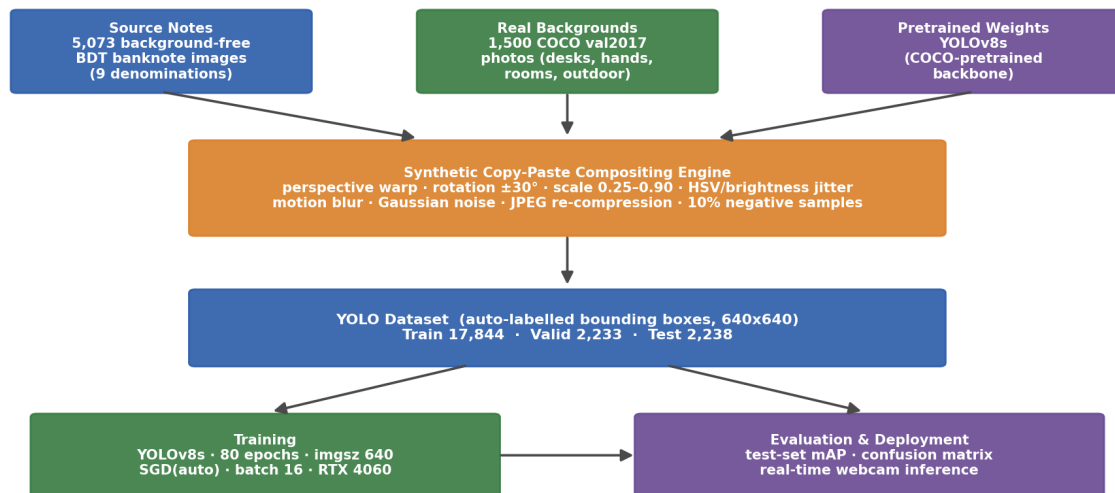


Figure 3 — End-to-end methodology pipeline.

4.1 Software Stack and Implementation

The system was implemented in Python. The compositing engine (`generate_synthetic_dataset.py`) uses Pillow and OpenCV for image transforms and NumPy for array maths. Training, validation and export use the **Ultralytics YOLOv8** framework (v8.4.64) on **PyTorch 2.6.0** with CUDA 12.4. The real-time application (`realtime_detect.py`) uses OpenCV for camera capture and display. All experiments ran on an NVIDIA RTX 4060 Laptop GPU (8 GB) with an AMD Ryzen 7 CPU under Windows 11. The project is organised as a set of reproducible scripts:

Script	Role
<code>download_backgrounds.py</code>	Fetch 1,500 COCO val2017 background images
<code>generate_synthetic_dataset.py</code>	Composite notes + backgrounds → labelled YOLO dataset
<code>train.py</code>	Fine-tune YOLOv8s with the augmentation recipe
<code>evaluate.py</code> / <code>random_test_check.py</code>	Test-set metrics, confusion matrix, spot checks
<code>realtime_detect.py</code>	Live webcam detection with low-light correction

Table 4 — Project scripts and their roles.

4.2 Detection Algorithm: YOLOv8

We use **YOLO (You Only Look Once)**, a *single-stage* object detector. Unlike two-stage detectors (e.g. Faster R-CNN) that first propose regions and then classify them, YOLO performs localisation and classification in a single forward pass over the image, which is what makes it fast enough for real-time video. The image is processed by a fully convolutional network that outputs, at three spatial resolutions, a

dense grid of predictions; each prediction carries a class distribution and four box-regression values.

YOLOv8 is **anchor-free**: rather than matching objects to a fixed set of pre-defined anchor-box shapes, it predicts box edges directly from each grid cell, which simplifies training and improves accuracy on unusual aspect ratios (relevant here, since banknotes are long and thin). Predictions are filtered by a confidence threshold and de-duplicated with **Non-Maximum Suppression (NMS)**, which keeps the highest-scoring box among heavily overlapping detections. We fine-tune the “s” (**small**) variant, which offers the best accuracy/speed trade-off for our hardware.

4.3 Model Architecture

YOLOv8s has three parts — a backbone, a neck and a head (Figure 4). Each is described below in plain terms.

Figure 2 — YOLOv8s Detection Architecture



11.1M parameters · 28.5 GFLOPs · 7.3 ms/image inference (RTX 4060)

Figure 4 — YOLOv8s detection architecture.

Input

The camera frame is resized to 640×640 pixels with three colour channels (RGB) and pixel values scaled to [0,1] before entering the network.

Backbone — CSPDarknet

The backbone is a convolutional feature extractor that progressively shrinks the image while deepening it, converting raw pixels into rich feature maps. It is built from **C2f** blocks (cross-stage partial modules that split, process and re-merge features for efficient gradient flow) and ends in an **SPPF** (Spatial Pyramid Pooling – Fast) module that pools over several receptive-field sizes so the network sees both fine detail and broad context. This is where the model learns what a banknote’s texture, portrait and numerals look like.

Neck — PAN-FPN

The neck fuses features from different depths of the backbone. A **Feature Pyramid Network (FPN)** carries high-level semantic information from deep layers down to shallow high-resolution layers, and a **Path Aggregation Network (PAN)** adds a bottom-up path back again. Together they let the detector recognise notes at very different sizes — a note filling the frame and a small note far from the camera are both handled well.

Head — Decoupled, Anchor-Free

The head makes the final predictions at three scales. It is **decoupled** — separate branches predict the class and the box — which improves accuracy, and **anchor-free** as described above. For our task it outputs, for every candidate location, a probability over the nine denomination classes plus four numbers

defining the box.

4.4 Loss Function

Training minimises a weighted sum of three losses that jointly teach the model *where* and *what* each note is:

- **Box regression (CIoU) loss** — penalises mismatch between predicted and true boxes using Complete-IoU, which accounts for overlap, centre distance and aspect ratio.
- **Distribution Focal Loss (DFL)** — refines box edges by predicting a distribution over positions rather than a single point, giving sub-pixel localisation.
- **Classification (BCE) loss** — binary cross-entropy over the nine classes, teaching the model which denomination occupies each detected box.

The default Ultralytics weightings were used (box 7.5, class 0.5, DFL 1.5).

4.5 Transfer Learning

Rather than training from scratch, we initialise from **COCO-pretrained weights**. The backbone has already learned general visual primitives (edges, textures, shapes) from millions of images, so fine-tuning only has to adapt these features to banknotes. This dramatically speeds convergence — the model exceeded 0.97 mAP@0.5 within eight epochs — and improves final accuracy compared with random initialisation, especially given a single-class-family task like ours.

4.6 Training Procedure

Training ran for 80 epochs at 640×640 with a batch size of 16 and the SGD optimiser (auto-configured by Ultralytics). On top of the offline compositing, YOLOv8's *online* augmentation was enabled — **mosaic** (stitching four images into one to vary context and scale) and **mixup** (blending two images) — along with HSV and geometric jitter. Horizontal and vertical flips were **disabled** because a note's orientation carries information and mirroring could corrupt printed text. Early-stopping patience was set to 15 epochs. The full configuration is given in Table 5.

Hyperparameter	Value	Hyperparameter	Value
Base model	yolov8s.pt	Optimizer	SGD (auto)
Epochs	80	Initial LR (lr0)	0.01
Image size	640×640	Final LR (lrf)	0.01
Batch size	16	Momentum	0.937
Early-stop patience	15	Weight decay	0.0005
Mosaic / Mixup	1.0 / 0.15	Rotation (deg)	20
HSV (h/s/v)	0.02/0.7/0.4	Flips (lr/ud)	0.0 / 0.0
Warmup epochs	3.0	Box/Cls/DFL gain	7.5 / 0.5 / 1.5

Table 5 — Key training hyperparameters.

5. Results and Analysis

5.1 Evaluation Metrics

Detectors are judged by **precision** (fraction of detections that are correct), **recall** (fraction of true notes found), and **mean Average Precision (mAP)**. mAP@0.5 counts a detection as correct if it overlaps the true box by at least 50% (Intersection-over-Union ≥ 0.5); the stricter **mAP@0.5:0.95** averages over IoU

thresholds from 0.5 to 0.95 and is far more sensitive to precise localisation — it is the headline number for real-world quality.

5.2 Overall Performance

Training converged rapidly: mAP@0.5 exceeded 0.97 by epoch 8 and 0.99 by epoch 11, after which mAP@0.5:0.95 continued to climb as localisation sharpened (Figure 5). Final metrics on the held-out test set (2,238 images) are given in Table 6.

Metric	Value
Precision	0.998
Recall	0.997
mAP@0.5	0.995
mAP@0.5:0.95	0.847
Inference speed	7.3 ms / image ($\approx$ 135 FPS)
Preprocess / Postprocess	1.7 ms / 1.7 ms

Table 6 — Test-set performance (best checkpoint).

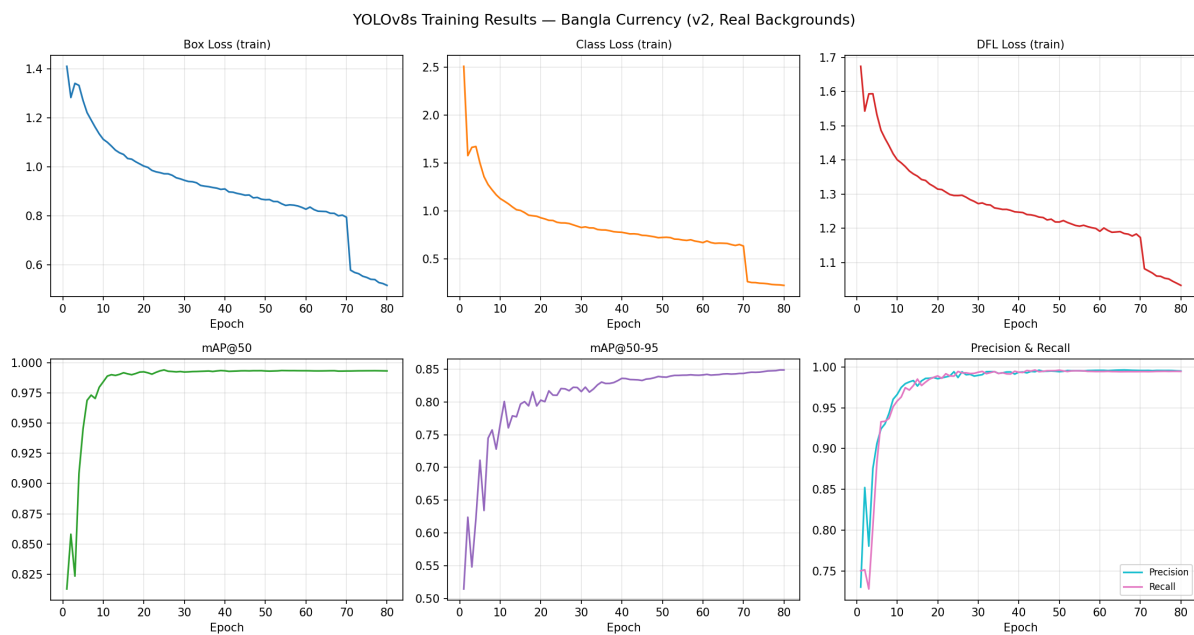


Figure 5 — Training curves: losses (top) and mAP / precision-recall (bottom) vs. epoch.

5.3 Per-Class Results

Performance is uniformly strong across all nine denominations, confirming the model does not favour high-frequency classes such as 500 Taka (which has the most source images).

Class	Images	Instances	Precision	Recall	mAP@0.5	mAP@.5:.95
2 Taka	180	180	0.998	1.000	0.995	0.838
5 Taka	264	264	0.998	0.996	0.995	0.844
10 Taka	172	172	0.995	0.988	0.995	0.848
20 Taka	192	192	0.998	1.000	0.995	0.847
50 Taka	172	172	1.000	0.993	0.995	0.851
100 Taka	228	228	0.999	1.000	0.995	0.845
200 Taka	172	172	0.999	1.000	0.995	0.843
500 Taka	500	500	1.000	1.000	0.995	0.853
1000 Taka	172	172	0.994	1.000	0.995	0.858
All	2238	2052	0.998	0.997	0.995	0.847

Table 7 — Per-class detection metrics on the test set.

5.4 Confusion Matrix

The normalised confusion matrix (Figure 6) is almost perfectly diagonal: every denomination is classified correctly with probability 0.99–1.00. The only residual error is minor confusion with the *background* column — occasional missed or spurious detections rather than denomination mix-ups. For a currency reader this is the safest failure mode, because the system never confuses one note value for another.

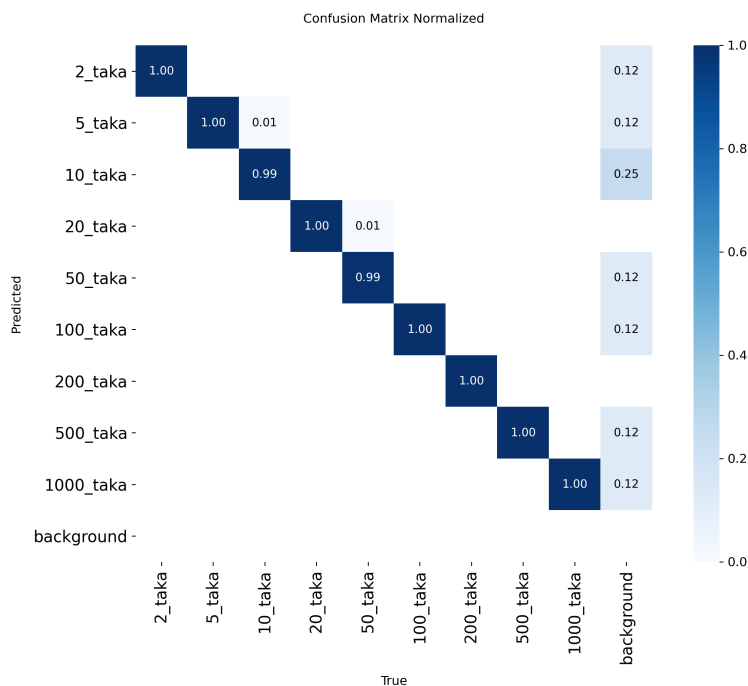


Figure 6 — Normalised confusion matrix on the test set.

5.5 Qualitative Detections

Figure 7 shows a validation batch with predicted boxes, labels and confidence scores. Notes are tightly localised across a range of scales, rotations and real backgrounds — exactly the conditions the compositing pipeline was designed to reproduce.

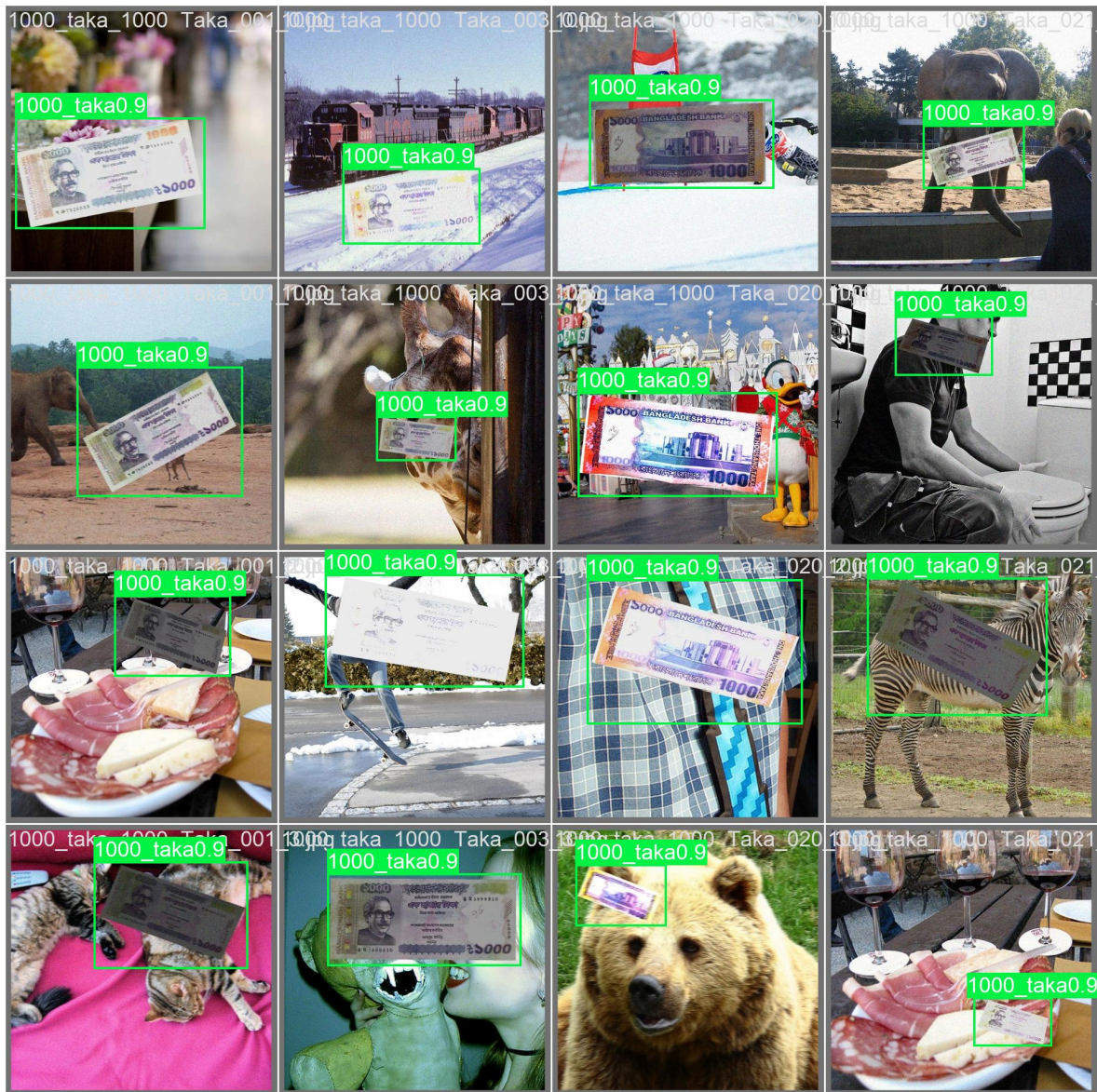


Figure 7 — Example predictions on a validation batch (label + confidence score).

6. Comparison with Similar Works

Table 8 situates our results among comparable Bangladeshi / multi-currency detection and recognition systems. Direct numerical comparison should be read with care because each study uses a different dataset, class set and test protocol; the table shows that our system is competitive with, or exceeds, published detection baselines while explicitly targeting real-world robustness.

Work	Approach	Task	Classes	mAP@0.5	mAP@.5:.95	P / R
NN + sym. masks [4]	Shallow NN	Classify	8	—	—	98.6% acc

CNN real-time [5]	Lightweight CNN	Classify	9	—	—	high acc
Dual-stream [7]	MobileNet+ EfficientNet	Classify	9	—	—	robust
YOLOv5 taka [1]	YOLOv5	Detect	—	—	—	0.95 / 0.93
Custom-head YOLOv8n [2]	YOLOv8n+SE	Detect	30	0.972	0.804	0.965 / 0.952
This work	YOLOv8s + synth. data	Detect	9 (BDT)	0.995	0.847	0.998 / 0.997

Table 8 — Comparison with related currency-recognition systems. “—” denotes a metric not reported by that work; “acc” = classification accuracy.

Several observations follow. First, our **mAP@0.5:0.95 of 0.847** — the metric most sensitive to localisation quality and real-world variation — is higher than the 0.804 reported by the custom-head YOLOv8n multi-currency detector [2], despite our use of an unmodified backbone; the gain is attributable to the domain-randomised training data rather than architectural tricks. Second, our precision and recall (0.998 / 0.997) exceed the YOLOv5 baseline's 0.95 / 0.93 [1]. Third, unlike the classification-only systems [4,5,7], our model outputs bounding boxes and therefore supports localisation and multi-note counting, not just single-note recognition. Finally, our labelled dataset (22,315 images) is far larger than the hand-annotated 3,111-image set in [1], obtained at zero annotation cost through automatic compositing.

7. Real-Time Deployment

The trained model is deployed in a webcam application built on OpenCV. Each frame is passed through the detector and annotated with live bounding boxes at interactive frame rates (≈ 135 FPS inference, well above real-time). A practical issue observed during testing was that low ambient light produced near-black frames (mean intensity $\approx 20/255$), which suppressed detections. The application therefore includes an **automatic brightness correction** step: when a frame's mean intensity falls below a threshold, a gain-and-gamma correction is applied before inference, restoring reliable detection in dim conditions. The model was additionally exported to **TorchScript** (.pts) so it can run without the Ultralytics Python dependency in production environments.

8. Limitations and Future Work

- **Synthetic realism.** Although real backgrounds and heavy augmentation greatly reduce the domain gap, composited notes still lack real cast shadows, physical folds and true occlusion. A modest set of genuine in-the-wild photographs for fine-tuning or validation would further strengthen real-world guarantees.
- **Condition and authenticity.** The system verifies denomination, not authenticity, and was trained on notes in good condition. Worn, torn or folded notes and counterfeit detection (cf. the JaalTaka benchmark) are natural extensions.
- **Edge deployment.** Future work includes INT8 quantisation and export to ONNX / TensorRT / TFLite for smartphone and embedded deployment, plus an audio-feedback mode to serve visually impaired users directly.

9. Conclusion

We presented a complete, reproducible pipeline for real-time Bangladeshi Taka detection that converts a clean classification dataset into a large, domain-randomised detection dataset and fine-tunes a YOLOv8s detector on it. The resulting model achieves 0.995 mAP@0.5 and 0.847 mAP@0.5:0.95 on a held-out test set with near-perfect precision and recall, runs above 100 FPS on a laptop GPU, and operates live from a webcam with automatic low-light correction. The central lesson is that *engineering the training data to match deployment conditions* — real backgrounds plus aggressive, physically motivated augmentation — is what turns a brittle studio-only classifier into a robust real-world detector.

References

- [1] YOLOv5-based Bangladeshi Taka detection subset (3,111 annotated images), reporting precision 0.95 and recall 0.93.
- [2] Real-Time Currency Detection and Voice Feedback for Visually Impaired Individuals, arXiv:2510.20267 — custom-head YOLOv8n (USD/EUR/BDT), mAP@0.5 0.972, mAP@0.5:0.95 0.804, 3.1 ms/image.
- [3] Nino, Gardi et al., “Synthetic-to-Real Object Detection using YOLO and Domain Randomization Strategies,” arXiv:2509.15045.
- [4] Bangladeshi Banknote Recognition by Neural Network with Axis-Symmetrical Masks — average accuracy 98.57% over eight denominations.
- [5] Bangladeshi Banknote Recognition in Real-time using Convolutional Neural Network for Visually Impaired People.
- [6] “A Diverse Image Dataset for Bangladeshi Currency Recognition” (BanglaTaka), Mendeley Data, 5,073 images, 9 denominations. <https://data.mendeley.com/datasets/3cv2syppkh/1>
- [7] Robust and Real-Time Bangladeshi Currency Recognition: A Dual-Stream MobileNet and EfficientNet Approach, arXiv:2602.07015.
- [8] Lin et al., “Microsoft COCO: Common Objects in Context,” ECCV 2014 (val2017 split used for backgrounds).
- [9] Jocher et al., Ultralytics YOLOv8 (v8.4.64). <https://github.com/ultralytics/ultralytics>